PingOne Privilege™ AI Gateway

Kubernetes Deployment Guide

High-availability deployment with an integrated load balancer

DEPLOYMENT GUIDE

Contents

1 Introduction and audience

2 Architecture overview

3 Prerequisites

4 Configuring the PingOne OIDC application

5 Deploying the first gateway

6 Shared Redis state store

7 Deploying the integrated load balancer (with HAProxy, Kong and F5 examples)

8 Adding a second gateway node

9 Registering MCP servers as applications

10 Verification

11 Operational considerations

12 Troubleshooting

Appendix A Resource summary

1 Introduction and audience

This guide describes how to deploy the PingOne Privilege AI Gateway on a Kubernetes cluster in a high-availability configuration. The deployment uses the official agentless-mcpgw Helm chart and includes an integrated load balancer that presents a single, stable URL to MCP clients while distributing traffic across gateway nodes.

Audience

This guide is intended for platform engineers and administrators who are familiar with:

- Kubernetes operations, including kubectl and namespaces
- Helm package management (Helm 3.14 or later is recommended)
- OAuth 2.0 / OpenID Connect concepts, particularly dynamic client registration
- The Model Context Protocol (MCP) and how AI agents consume MCP servers

What this deployment provides

- One public URL for every MCP client and application — no client-side changes when gateways are added or removed.
- Gateway high availability — multiple gateway nodes serve the same mesh cluster; a failed node is removed from rotation automatically.
- Durable client registrations — a shared Redis store preserves dynamic client registrations across gateway restarts and failovers.
- Centralized policy enforcement — per-application authorization policies, guardrails and audit logging remain in force regardless of which node serves a request.

> **NOTE**
> This guide reflects the tested reference deployment. Where behavior was verified against a live cluster, it is stated as such; items that depend on your environment are marked as environment-specific.

2 Architecture overview

Request path

```
MCP client / AI agent (browser, IDE, Postman, LM Studio)
        |
        |  https://mcpgw.example.com/<AppName>/mcp
        v
Ingress controller (TLS termination, public hostname)
        |
        v
mcpgw-lb service :8623   (integrated load balancer, 2 replicas)
        |    consistent-hash pinning: client address + application path
        v
Gateway node 1 ---- Gateway node 2 ---- Gateway node N
   (all nodes are members of ONE mesh cluster)
        |
        v
MCP backends: sidecar containers (localhost) or cluster services
```

Components

Component	Role	Notes
Gateway (agentless-mcpgw)	Serves MCP traffic, OAuth flows, policy enforcement and audit	Deployed with the official Helm chart; one release per node
Mesh cluster	Logical grouping of gateway nodes in the Privilege console	Applications are registered once per cluster; every node serves them
Load balancer (mcpgw-lb)	Single entry point; pins each client + application to one node	Ships as part of the Helm chart; HAProxy-based health checks
Shared Redis	Stores dynamic client registrations	Survives gateway restarts and failover; tokens are not shared
Sidecar MCP servers	Backends running inside the gateway pod on localhost ports	Required for catalog applications pinned to http://localhost:8080/mcp

Session behavior and failover

Each gateway node holds issued access tokens in memory. The load balancer therefore pins every client, for every path of a given application, to the node that authenticated it. On failover:

- Client registrations survive (shared Redis) — clients are still known to the other node.
- Client tokens do not survive — affected clients re-authenticate automatically; with single sign-on this typically takes under 30 seconds and usually requires no prompt.
- The load balancer holds a failed or restarting node out of rotation until it passes a configurable number of consecutive health checks.

> **NOTE**
> Round-robin load balancing across gateway nodes is not supported: a token issued by one node is rejected by the others. The integrated load balancer enforces pinning automatically — do not replace it with a generic round-robin service.

3 Prerequisites

Requirement	Details
Kubernetes cluster	Version 1.25 or later; at least 2 worker nodes for HA
Helm	3.12 or later
Ingress controller	nginx (or equivalent) with TLS termination and a wildcard or per-host certificate
Certificate issuer	cert-manager with a Let's Encrypt (or corporate CA) ClusterIssuer
DNS	A public hostname for the gateway URL, published from the ingress (for example via external-dns)
PingOne environment	A PingOne environment with the PingOne Privilege service enabled, tenant administrator access
Container registry access	Image pull secret for the gateway and sidecar images

Environment checks

```
kubectl config current-context          # confirm the intended cluster
kubectl get clusterissuer               # a Ready issuer must exist
kubectl get ingressclass                # note the default ingress class
helm version
```

> **NOTE**
> Replace all hostnames, namespaces and identifiers in this guide with your own values. The examples use `mcpgw.example.com` and the namespace `privilege-gw`.

4 Configuring the PingOne OIDC application

The gateway signs users in through an OIDC application in your PingOne environment. Create it in the PingOne admin console before deploying the gateway.

1. In the PingOne admin console, go to Applications and click + Add Application.

2 Select Web Application. Note the generated client ID and client secret.
3 On the Configuration tab, set Token Endpoint Authentication Method to Client Secret Basic.
4 Set the redirect URI to `https://mcpgw.example.com/callback`.
5 On the Grants tab, enable Authorization Code and Token Exchange.
6 On the Resources tab, grant `openid`, `profile` and `email`.
7 Save the application. Record the environment ID — it appears in your OIDC endpoints.

IMPORTANT
Do not enable PKCE-only or public-client settings on this application. The gateway authenticates to the token endpoint with the client secret (Client Secret Basic).

5 Deploying the first gateway

5.1 Generate an enrollment token

1 In the Privilege console, open Gateways and click Add New.
2 Choose the AI GW form (the Docker command option).
3 Enter a Cluster ID — this names your mesh cluster (for example `production-cluster`). Applications reference this ID.
4 Enter the Host IP: the node's own in-cluster Kubernetes Service name (see section 8).
5 Click Get Docker Command and copy the value of `ENV_PROXY_TOKEN` (the string beginning `eyJ`).

IMPORTANT
Enrollment tokens are single use and expire after approximately two hours. Strip any leading or trailing whitespace before saving the token — whitespace corrupts the signature check and crash-loops the gateway. Store the token in a file with owner-only permissions and delete it after use.

5.2 Create the secrets

```
cat > gw-secrets.yaml <<'YAML'
proxyToken: "eyJ..."            # enrollment token from the console
oidc:
  clientId: "<client ID>"
  clientSecret: "<client secret>"
  authUrl:  "https://auth.pingone.com/<environmentId>/as/authorize"
  tokenUrl: "https://auth.pingone.com/<environmentId>/as/token"
  userUrl:  "https://auth.pingone.com/<environmentId>/as/userinfo"
YAML
kubectl create namespace privilege-gw
kubectl -n privilege-gw create secret generic gw-oidc --from-file=gw-secrets.yaml
rm gw-secrets.yaml
```

5.3 Install the chart

```
helm upgrade --install agentless-mcpgw <chart>.tgz \
  --namespace privilege-gw \
  --set hostname=mcpgw.example.com \
  --set namespace=privilege-gw \
  --set oidc.serverUrl=https://mcpgw.example.com \
  --set-file proxyToken=<token-file> \
  -f sidecars.values.yaml
```

The enrollment is persisted in the gateway's SSL persistent volume, so subsequent pod restarts do not need a new token.

5.4 Confirm enrollment

```
kubectl -n privilege-gw exec deploy/agentless-mcpgw -c log-tailer -- \
  tail -100 /var/log/procyon/cyonproxy.log | grep -E 'ClusterID|LinkStatus'
```

Success is indicated by `ClusterID:<your cluster>` and `LinkStatus:Active`. Messages of the form `has same NodeURL` are cosmetic and can be ignored.

> **NOTE**
> The gateway writes its log to a file rather than stdout. Read it with the log-tailer sidecar as shown above.

5.5 Changing OIDC configuration

OIDC settings are read from the secret at pod start. After any change to OIDC values, restart the gateway pod:

```
kubectl -n privilege-gw rollout restart deploy/agentless-mcpgw
```

6 Shared Redis state store

Gateway nodes keep dynamic client registrations in a Redis store when `MCPGW_REDIS_URL` is configured. Without it, every gateway restart forgets all registered clients and each client must register again. Deploy Redis before adding gateway nodes, because a gateway that cannot reach Redis at startup exits after six attempts.

```
# 1. Password secret and Redis itself
kubectl -n privilege-gw create secret generic privilege-redis \
  --from-literal=password="$(openssl rand -base64 24 | tr -d '/+=')"
kubectl apply -f redis.yaml          # 1Gi PVC, append-only persistence recommended

# 2. URL secret (value is never printed)
URL="redis://:$(kubectl -n privilege-gw get secret privilege-redis \
  -o jsonpath='{.data.password}' | base64 -d)@privilege-redis.privilege-gw.svc.cluster.local:6379/0"
kubectl -n privilege-gw create secret generic privilege-redis-url --from-literal=url="$URL"

# 3. Enable on the gateway release
helm upgrade agentless-mcpgw <chart>.tgz -n privilege-gw --reuse-values \
  --set redis.urlSecret.name=privilege-redis-url
```

Rule	Reason
Never restart a gateway while Redis is down	The gateway retries six times, then exits; the pod crash-loops until Redis returns
Use the environment variable name MCPGW_REDIS_URL	A bare REDIS_URL is ignored by the gateway
Persist Redis on a volume (append-only file)	A Redis restart then preserves registrations
Keep Redis a single, small deployment	It is a state store only; size 64 MB maxmemory is sufficient for registrations

Verify with the log line `Redis token store connected on attempt 1/6` in the gateway log, and by confirming that Redis holds keys after a client registration.

7 Deploying the integrated load balancer

The load balancer is part of the Helm chart (`templates/loadbalancer.yaml`) and is enabled on the primary gateway release. It provides:

- Consistent-hash pinning — the routing key is the client address plus the first path segment (the application). One client's calls to one application always reach the same node while that node is healthy.
- Callback routing by state tag — OAuth callbacks carry a tag identifying the issuing node, so sign-in completes on the node that started it.
- Active health checks — each node is probed every 10 seconds; a failed node is held out of rotation for a configurable failover timeout (default 600 seconds).
- Visibility — every response carries an `X-Served-By` header naming the node that answered; an optional statistics page is available behind basic authentication.

```
helm upgrade agentless-mcpgw <chart>.tgz -n privilege-gw --reuse-values \
  -f sidecars.values.yaml -f lb.values.yaml
```

Value	Purpose
loadBalancer.enabled	Enables the load balancer; the ingress routes to it automatically
loadBalancer.failTimeout	How long a failed node is held out of rotation (default 600s)
loadBalancer.peers[*].parked	Removes a node from rotation (maintenance); restore with parked: false
loadBalancer.ownParked	Parks the local node (used to test the remaining nodes)
loadBalancer.stats.auth	user:password for the /lb-stats page; unset leaves the page at 401
loadBalancer.runtimeSocket.enabled	Enables runtime park/unpark through an admin API

IMPORTANT
After any load balancer rollout or gateway restart, a node re-enters rotation only after 60 consecutive successful health checks (approximately 10 minutes with the default settings). During that window clients receive 503 responses with `node=<NOSRV>`. This is expected behavior, not an outage — do not restart the gateway in response, as that resets the timer. Time deployments accordingly.

NOTE
Always upgrade with the same values files (for example `sidecars.values.yaml`) and `--reuse-values`. An upgrade without them resets sidecar containers and routing to chart defaults.

7.1 Routing requirements for any load balancer

If you front the gateways with your own load balancer instead of the integrated one, it must implement the following behavior. A conventional round-robin HTTP load balancer is not sufficient: each gateway node holds issued access tokens in memory, so a client's sign-in, token exchange and MCP calls must all reach the same node.

Requirement	Why
Pin by client address + application path (first path segment)	Sign-in state and tokens are per node. One client's /<App>/register, /authorize, /token and /mcp calls must land on the same node while it is healthy.
Route the global /callback by OAuth state	The callback arrives without the application in the path. The authorize redirect appends the issuing node to the state parameter; the callback must be routed back to that node. With the integrated load balancer this is automatic.

Requirement	Why
Long timeouts on streaming paths	MCP uses long-lived HTTP streams. Configure read/send timeouts of at least 3600 seconds and disable response buffering on the application paths.
Active health checks, not passive only	Probe each node (GET /, any 2xx-4xx counts as up). Hold a failed node out until it passes consecutive checks; a short return window forces clients to re-authenticate twice.
Retry to a surviving node on connection failure	A request that cannot connect should be retried once on another node; the affected client then re-authenticates automatically.

7.2 Example: HAProxy

HAProxy is the engine the integrated load balancer uses (set `loadBalancer.engine: haproxy`). A standalone equivalent:

```
haproxy
frontend fe
    bind :443 ssl crt /etc/haproxy/certs/gateway.pem
    # pin: client address + first path segment (the application)
    http-request set-var(txn.app) path extracted_string,word(1,/)
    http-request set-var(txn.pin) concat(src,'|',txn.app)
    default_backend gateways

backend gateways
    balance hash var(txn.pin)
    # gateway 1 speaks HTTPS (its OAuth URL matches its certificate);
    # additional nodes answer plain HTTP and are reached over the bridge
    option httpchk GET /
    default-server check inter 10s fall 3 rise 60
    server gateway-1 agentless-mcpgw.privilege-gw.svc.cluster.local:443 \
        check check-sni mcpgw.example.com ssl verify none
    server gateway-2 127.0.0.1:8444 check

# OAuth callback: route to the node named in the state tag
# (the authorize redirect appends ~gateway-N to the state parameter;
# strip the tag before passing the request on)
http-request use-service lua.reply-missing if { path /callback }
```

NOTE

The state-tag callback routing and the per-node TLS bridge are handled inside the integrated load balancer's generated configuration. Reproducing them in a standalone HAProxy requires the same two mechanisms: tag the authorize 302 with the issuing node, and route untagged traffic by hash only.

7.3 Example: Kong

Kong Gateway can front the gateways as services with hash-based upstream balancing. Configure one upstream with consistent hashing on the pin key:

```
# 1. Upstream with consistent hashing
curl -X POST http://localhost:8001/upstreams \
  --data name=gateway-upstream \
  --data algorithm=consistent-hashing \
  --data hash_on=header \
  --data hash_fallback=none

# 2. Targets (one per gateway node)
curl -X POST http://localhost:8001/upstreams/gateway-upstream/targets \
  --data target=agentless-mcpgw.privilege-gw.svc.cluster.local:443
curl -X POST http://localhost:8001/upstreams/gateway-upstream/targets \
  --data target=agentless-mcpgw2.privilege-gw2.svc.cluster.local:8623

# 3. Active health checks (a gateway answering 401 to GET / is healthy)
curl -X PATCH http://localhost:8001/upstreams/gateway-upstream \
  --data healthchecks.active.type=http \
  --data healthchecks.active.http_path=/ \
  --data healthchecks.active.healthy.interval=10 \
  --data healthchecks.active.healthy.successes=2 \
  --data healthchecks.active.unhealthy.interval=10 \
  --data healthchecks.active.unhealthy.http_failures=3

# 4. Service + route with long streaming timeouts
curl -X POST http://localhost:8001/services \
  --data name=gateway \
  --data host=gateway-upstream \
  --data protocol=https \
  --data connect_timeout=5000 \
  --data read_timeout=3600000 \
  --data write_timeout=3600000 \
  --data retries=1

curl -X POST http://localhost:8001/routes \
  --data 'paths[]=/' \
  --data service.name=gateway
```

IMPORTANT

Hash on a pin key that includes the client address and the application path (a pre-function plugin can compute it into a header). Plain consistent-hashing on the source IP alone is close but not equivalent: the callback arrives without the application in the path, and clients behind a shared NAT all hash to one node.

7.4 Example: F5 BIG-IP

On BIG-IP, reproduce the requirements with a performance L4 policy or an iRule. The essential elements:

```
# Pool with two gateway nodes; HTTPS monitor treats any 2xx-4xx as up
tmsh create ltm pool pgw-pool \
  members add { agentless-mcpgw.privilege-gw.svc.cluster.local:443 \
                agentless-mcpgw2.privilege-gw2.svc.cluster.local:8623 } \
  monitor /Common/tcp

# Long client- and server-side timeouts for streaming MCP traffic
tmsh create ltm profile tcp pgw-tcp-l4 \
  idle-timeout 3600

tmsh create ltm virtual pgw-vs \
  destination 10.0.0.5:443 \
  ip-protocol tcp \
  pool pgw-pool \
  profiles add { pgw-tcp-l4 tcp } \
  persist replace { ltm persistence source-addr { \
      defaults-from /Common/source-addr \
      timeout 3600 } }
```

BIG-IP element	Maps to requirement
Source-address persistence (3600 s)	Pins one client to one node; combine with a path-scoped iRule for per-application pinning
L4 (performance) virtual server	Streams pass through unbuffered; HTTP inspection would re-buffer long MCP responses
TCP monitor on the pool	Active health checking; use an HTTP monitor against / if the gateway's 401-on-GET behavior is acceptable to the monitor
Connection mirroring / HA pair	Failover of the load balancer itself; gateway pinning state is in the client, not the LB

NOTE

Whatever the platform, validate the full flow before production: register (201), authorize (302), callback (302), token (200), then repeated /mcp calls (200) — all answered by the same node (X-Served-By header with the integrated load balancer; add an equivalent header when rolling your own). Then kill one node and confirm the remaining node keeps serving after a single re-authentication.

8 Adding a second gateway node

For high availability, add at least one more gateway node to the same mesh cluster. Each node serves every application registered on the cluster; no application registration is duplicated.

8.1 Enroll the node

1 In the Privilege console, open Gateways and click Add New, then choose the AI GW form.

2. Set Cluster ID to the existing cluster name — this makes the new deployment a second node rather than a new cluster.
3. Set the Host IP to the new node's in-cluster Kubernetes Service name, for example `agentless-mcpgw2.privilege-gw2.svc.cluster.local`. Do not accept a pre-filled public hostname: a node enrolled with the wrong Host IP never appears in the console and cannot be edited later — it must be re-created.
4. Generate the Docker command and copy the enrollment token as in section 5.1.

8.2 Install the second release

Install the chart a second time, in a second namespace, with a distinct release name and `nameOverride` so resource names do not collide. Set `ingress.enabled: false` — the load balancer, not the ingress, routes to additional nodes. Include the same sidecar values file so both pods run the same sidecar set.

```
# Move any prior enrollment files aside before re-enrolling:
kubectl -n privilege-gw2 scale deploy/agentless-mcpgw2 --replicas=0
kubectl -n privilege-gw2 delete pvc agentless-mcpgw-ssl     # old enrollment only

helm upgrade --install agentless-mcpgw2 <chart>.tgz \
  --namespace privilege-gw2 \
  --set nameOverride=agentless-mcpgw2 \
  --set ingress.enabled=false \
  --set hostname=mcpgw.example.com \
  --set oidc.serverUrl=https://mcpgw.example.com \
  --set-file proxyToken=<token-file> \
  -f sidecars.values.yaml
```

8.3 Synchronize secrets and state

- Copy the sidecar secrets into the second namespace (each sidecar's credentials, plus the image pull secret and the Redis URL secret).
- Enable the shared Redis URL secret on the second release (section 6).
- Confirm `Redis token store connected` and `LinkStatus:Active` in the new node's log before admitting traffic.

8.4 Admit the node to rotation

Add the node to the load balancer's peer list in the primary release's values file and upgrade. Remove it with `parked: true`. Define peer entries in the values file — do not modify list values with `--set` indexes on an existing release.

9 Registering MCP servers as applications

An MCP server is exposed to clients as an application in the Privilege console. Register each server once, against your mesh cluster; every gateway node serves it.

9.1 Register the application

1. In the Privilege console, go to Agentic Apps and click Add Application → MCP Server.
2. Enter an Application Name. It becomes the client-facing path: `https://mcpgw.example.com/<AppName>/mcp`.
3. Enter the MCP Server URL — the backend endpoint, for example `http://my-mcp-server.privilege-gw.svc.cluster.local/mcp` for a cluster service, or `http://localhost:<port>/mcp` for a sidecar.
4. Select the Mesh Cluster (the cluster ID from section 5.1).
5. Set Auth Mode as appropriate (typically None for internal backends).
6. Save. The gateway discovers the server by sending an MCP initialize request to the registered URL.

IMPORTANT
Register backends with the /mcp path, not /sse. Current gateway builds discover servers by POSTing an initialize request; an SSE endpoint rejects that with HTTP 405 and discovery fails. Also note that the entry path is stored at registration time: correcting a wrong path requires deleting and re-registering the application — a restart does not recompute it.

9.2 Sidecar backends

Applications created from the MCP catalog are pinned to `http://localhost:8080/mcp` and can only be served by a sidecar container inside the gateway pod. Add sidecars through the chart's `extraContainers` value: give each server a distinct local port (8080–8082 are used by the standard sidecars), a readiness probe on `/health`, and register it in the console as its own application. Every gateway node must run the same sidecar set, with the sidecars' secrets present in every node's namespace.

9.3 Authorization policies

Policies are defined per application and reference synced users and groups. A newly registered application denies all access until a policy covering the intended users is created. When a call is denied, the reason is written to the gateway log — the client sees only a generic 403.

10 Verification

10.1 Node routing and pinning

```
for i in 1 2 3 4; do
  curl -sk -o /dev/null -D - https://mcpgw.example.com/<AppName>/mcp | grep -i x-served-by
done
```

Every request from one client address and application path should return the same node name. Requests from different networks may map to different nodes.

10.2 Load balancer logs

```
kubectl logs -l app=mcpgw-lb --prefix --max-log-requests=4 \
  | grep -E 'node=|register|authorize|token'
```

Each client request is logged with the serving node. The line from the ingress address carries the authoritative `node=` value.

10.3 End-to-end client test

1. In an MCP client (for example an IDE extension, Postman or LM Studio), add a server with only the URL `https://mcpgw.example.com/<AppName>/mcp` and no static credentials.
2. The client performs dynamic client registration and opens a browser for the PingOne sign-in.
3. Complete the sign-in and run a tool call.
4. In the load balancer log, confirm the sequence register (201) → authorize (302) → callback (302) → token (200) → mcp (200).

10.4 Failover test

1. Sign in a client and confirm which node serves it (X-Served-By header).
2. Remove that node from rotation (parked: true) or scale its deployment to zero.
3. The client's traffic moves to a surviving node within one health-check interval; the client re-authenticates automatically (registrations are preserved by Redis; tokens are not).
4. Restore the node and confirm it re-enters rotation after the health-check rise window.

11 Operational considerations

Chart hygiene

- If you customize chart templates, repackage the chart (`helm package`) in the same change. Deploying from a stale package silently drops template-level additions such as sidecar containers and image pull secrets.
- Before any upgrade that touches sidecars, render a dry run and verify the sidecar containers appear in the output.
- Upgrades of a running release should use `--reuse-values` and the same overlay files; a bare upgrade resets sidecars, Redis settings and routing.
- Avoid modifying nested list values with `--set` indexes; use a values file that restates the full list.

Restart discipline

- Never restart a gateway while Redis is unavailable — the gateway exits after six failed connection attempts and crash-loops until Redis returns.
- A gateway restart costs clients its sessions and takes the node out of LB rotation for roughly 10 minutes (health-check rise window). Schedule restarts.
- OIDC configuration changes take effect only after a pod restart.

Capacity

- The main gateway container idles at very low CPU; the chart default request of 1 CPU may be reduced (for example to 250m) on constrained clusters while keeping the limits unchanged.

- Each sidecar adds to the pod footprint; a pod with eight sidecars is a nine-container pod. Plan node resources accordingly.
- Redis with a 1Gi volume and 64 MB maxmemory is sufficient for client registrations.

Security

- Enrollment tokens, OIDC client secrets and generated values files are sensitive: store them outside source control, delete token files after use, and rotate any credential that has been exposed in screenshots or chat history.
- Restrict the load balancer statistics page with authentication, or disable it.
- The Redis connection is password-authenticated in-cluster; if your security model requires TLS for data stores, front Redis accordingly.

12 Troubleshooting

Symptom	Likely cause	Resolution
Console shows “Gateway Unreachable — Error discovering MCP server … Unauthorized”	Backend registered on /sse, wrong hostname, or expired policy — read the gateway log line “Error discovering MCP server” for the actual method and path	Re-register the backend with /mcp; verify the URL resolves from the gateway pod
Client receives “Unknown client” at /authorize	The gateway forgot the DCR registration (Redis unavailable or not configured)	Restore Redis, then have the client register again; verify MCPGW_REDIS_URL
401 on //mcp immediately after a successful sign-in	The token came from one node but the call reached another (pinning bypassed)	Ensure clients enter through the load balancer, not a node Service directly
500 on //mcp while sign-in works	The node serving the call lacks the sidecar the backend runs on	Apply the same sidecar values file and secrets to every node
503 node=<NOSRV> from the public URL	All backends are in the health-check rise window after a restart or rollout	Wait for the rise window (up to ~10 minutes); do not restart gateways
Crash loop with “crypto/ecdsa: verification error” at enrollment	The enrollment token contained leading/trailing whitespace	Re-create the token, strip whitespace, delete the old SSL PVC, re-enroll
Authorize redirects with an empty client_id	The pod is still running pre-upgrade OIDC configuration	Restart the gateway pod after OIDC secret changes
An application stops working after a console edit	Its Mesh Cluster was changed, which moves the application off the original cluster	Set the Mesh Cluster back to the production cluster
Gateway pod exits repeatedly, log shows fatal “unreachable after 6 attempts”	Redis was down when the gateway restarted	Restore Redis first; the gateway reconnects automatically once Redis is reachable

Reading the gateway log

```
kubectl exec deploy/agentless-mcpgw -c log-tailer -- \
  tail -200 /var/log/procyon/cyonproxy.log
```

Policy denials are logged with the resolved user and the reason (for example, the user is not synced into Privilege, or no policy on the application covers them). The client sees only a generic 403, so always confirm in the log.

Appendix A Resource summary

Resource	Namespace	Name	Purpose
Gateway deployment	privilege-gw	agentless-mcpgw	Primary gateway node
Gateway deployment	privilege-gw2	agentless-mcpgw2	Second node (nameOverride, no ingress)
SSL PVC	each node	agentless-mcpgw-ssl	Enrollment state; delete only to re-enroll
Load balancer	privilege-gw	mcpgw-lb (Deployment + Service)	Client entry point, 2 replicas
Ingress	privilege-gw	gateway ingress	TLS termination for the public hostname
Redis	privilege-gw	privilege-redis (Deployment, Service, 1Gi PVC)	Shared registration store
Secrets	every namespace	privilege-redis, privilege-redis-url, sidecar secrets, pull secret	Credentials for Redis and sidecars
OIDC secret	privilege-gw	gw-oidc	Enrollment token and OIDC configuration

Ping Identity Proprietary and Confidential. This document describes a reference deployment and is provided as-is. Values shown are examples; substitute your own hostnames, namespaces and identifiers.